

Sistema de Gestão de Tarefas com Arquitetura de Microsserviços

João Silva
Universidade da Beira Interior
Covilhã, Portugal

João Patrocínio
Universidade da Beira Interior
Covilhã, Portugal

João Maravilhoso
Universidade da Beira Interior
Covilhã, Portugal

Resumo—Este relatório descreve o desenvolvimento de um sistema de gestão de tarefas baseado numa arquitetura de microsserviços. O sistema foi implementado em Python com a framework Flask, utilizando MariaDB como base de dados. A comunicação entre serviços segue o modelo REST (Request/Response) e a arquitetura adotada é a de orquestração, com um serviço central que encaminha todos os pedidos. Cada microsserviço corre no seu próprio contentor Docker e o sistema está integrado num ambiente local Kubernetes através do Minikube.

I. INTRODUÇÃO

O objetivo deste projeto é o desenvolvimento de um sistema para gestão de tarefas, composto por vários microsserviços independentes. A ideia é que cada parte do sistema tenha a sua própria responsabilidade, estando separada das restantes, o que facilita a manutenção e a escalabilidade.

O sistema permite que utilizadores se registem, façam login, e a partir daí possam criar, editar, listar e apagar as suas tarefas. Além das funcionalidades mínimas exigidas, implementámos também um serviço de notificações que avisa os utilizadores quando o prazo de uma tarefa está próximo, e uma interface web para interação com o sistema.

Ao longo deste relatório, vamos explicar como modelámos os serviços, que decisões técnicas tomámos e porquê, e como fizemos o *deployment* em Docker e Kubernetes.

II. MODELAÇÃO DOS SERVIÇOS

A. Definição do Domínio

Antes de começar a programar, tentámos perceber qual era o domínio do nosso problema. Seguindo os conceitos abordados nas aulas, em particular a noção de *bounded contexts*, identificámos as diferentes áreas de funcionalidade do sistema.

O domínio é a “gestão de tarefas”. Dentro deste domínio, conseguimos identificar os seguintes contextos delimitados:

- **Contexto de Autenticação** – Tudo o que diz respeito ao registo e login dos utilizadores, geração e validação de tokens JWT.
- **Contexto de Utilizadores** – Gestão das contas de utilizador (criar, editar, listar, apagar).
- **Contexto de Tarefas** – Gestão das tarefas propriamente ditas, incluindo estados e prazos.
- **Contexto de Notificações** – Criação e gestão de notificações associadas a prazos de tarefas.

Cada um destes contextos foi mapeado para um microsserviço independente. A comunicação entre eles é feita apenas através de interfaces REST bem definidas, garantindo um acoplamento fraco entre os serviços.

B. Diagrama da Arquitetura

A Fig. 1 mostra a arquitetura geral do sistema. Todos os pedidos externos passam pelo serviço orquestrador, que os reencaminha para o microsserviço correto.

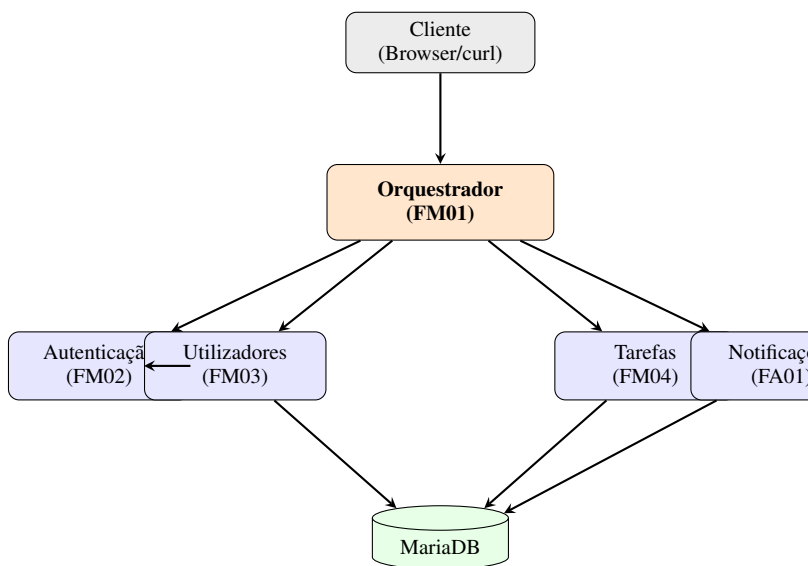


Figura 1. Diagrama da arquitetura do sistema.

C. Modelo de Domínio

A Fig. 2 apresenta o modelo de domínio simplificado, mostrando as entidades principais e as relações entre elas.

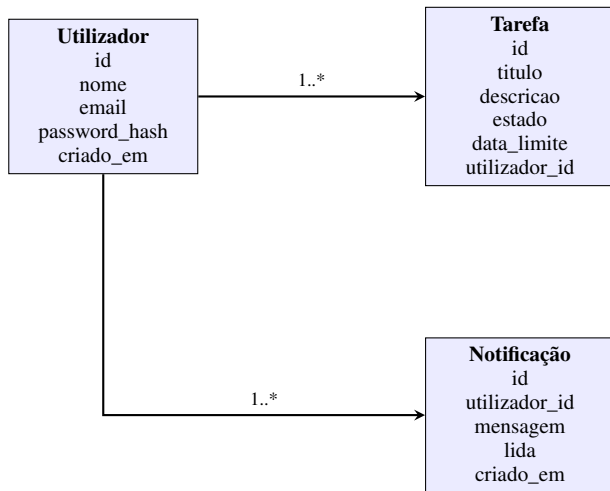


Figura 2. Modelo de domínio do sistema.

Um utilizador pode ter várias tarefas e várias notificações. Cada tarefa pertence a um único utilizador. As notificações são geradas automaticamente quando há tarefas com prazos próximos.

III. DESCRIÇÃO DOS MICROSERVIÇOS

A. FM01 – Serviço Orquestrador

O orquestrador é o ponto de entrada do sistema. Corre na porta 5000 e todas as chamadas externas passam por ele. As suas responsabilidades são:

- Receber os pedidos HTTP dos clientes.
- Verificar a autenticação (extrair e validar o token JWT no cabeçalho *Authorization*).
- Reencaminhar o pedido para o microserviço adequado.
- Verificar permissões (por exemplo, um utilizador só pode editar ou apagar as suas próprias tarefas).
- Disponibilizar um endpoint `/api/saude` que verifica o estado de todos os serviços.

Optamos por esta abordagem de orquestração centralizada porque simplifica a lógica do lado do cliente: basta conhecer o endereço do orquestrador. Além disso, permite-nos centralizar a validação de tokens num único ponto, em vez de ter cada serviço a fazer essa verificação.

B. FM02 – Serviço de Autenticação

Este serviço é responsável pelo registo e login dos utilizadores. Corre na porta 5001 e expõe três endpoints:

- `POST /auth/registar` – Cria uma nova conta (chama internamente o serviço de utilizadores) e devolve um token JWT.
- `POST /auth/login` – Autentica o utilizador verificando as credenciais e devolve um token JWT.
- `POST /auth/verificar` – Valida um token JWT (usado internamente pelo orquestrador).

Utilizamos a biblioteca `PyJWT` para gerar e verificar os tokens. Cada token tem validade de 24 horas e contém o ID, nome e email do utilizador no *payload*. As senhas são

guardadas com hash gerado pela biblioteca `extttWerkzeug` e nunca em texto limpo.

C. FM03 – Serviço de Utilizadores

O serviço de utilizadores gere as contas na base de dados e corre na porta 5002. Implementa as operações CRUD completas:

- `POST /utilizadores` – Criar utilizador.
- `GET /utilizadores` – Listar todos.
- `GET /utilizadores/<id>` – Obter por ID.
- `GET /utilizadores/email/<email>` – Obter por email (rota interna usada pelo serviço de autenticação).
- `PUT /utilizadores/<id>` – Atualizar dados.
- `DELETE /utilizadores/<id>` – Eliminar conta.

Este serviço é o único que lida diretamente com os dados dos utilizadores. O serviço de autenticação comunica com ele via HTTP para criar contas e verificar credenciais.

D. FM04 – Serviço de Tarefas

Responsável pela gestão das tarefas criadas pelos utilizadores. Corre na porta 5003 e disponibiliza:

- `POST /tarefas` – Criar tarefa.
- `GET /tarefas` – Listar tarefas (com filtro opcional por `utilizador_id`).
- `GET /tarefas/<id>` – Obter tarefa específica.
- `PUT /tarefas/<id>` – Atualizar tarefa.
- `DELETE /tarefas/<id>` – Eliminar tarefa.

Cada tarefa tem um título, descrição (opcional), um estado que pode ser *pendente*, *em_progresso* ou *concluida*, e opcionalmente uma data limite.

E. FA01 – Serviço de Notificações (Extra)

Este serviço é uma funcionalidade extra. Corre na porta 5004 e tem dois mecanismos:

- 1) Quando o orquestrador cria uma tarefa com data limite, envia um pedido ao serviço de notificações para registar a notificação.
- 2) Uma *thread* em segundo plano verifica a cada 60 minutos se existem tarefas com prazo a expirar no dia seguinte e, se existirem, cria automaticamente notificações para os respetivos utilizadores.

Os utilizadores podem listar as suas notificações e marcá-las como lidas.

F. FA02 – Interface Web (Extra)

Implementámos uma interface web simples em HTML, CSS e JavaScript puro, servida por um contentor Nginx. A interface permite:

- Registo e login de utilizadores.
- Visualização das tarefas com código de cores por estado.
- Criação, edição e eliminação de tarefas através de modais.
- Painel de notificações com contagem de não lidas.

A comunicação com a API é feita via `fetch` para o orquestrador, enviando o token JWT no cabeçalho de cada pedido autenticado.

IV. DECISÕES TÉCNICAS

A. Linguagem e Framework

Escolhemos Python com Flask por ser uma combinação que já conhecíamos das aulas e que nos permite criar APIs REST de forma rápida e sem muita complexidade. O Flask é leve e não impõe uma estrutura rígida, o que nos deu liberdade para organizar o código como queríamos.

B. Base de Dados

Optámos pelo MariaDB por ser relacional e por estar entre as opções sugeridas no enunciado. A base de dados é partilhada entre os serviços que precisam dela (utilizadores, tarefas e notificações), mas cada serviço acede apenas às suas próprias tabelas. Numa versão mais avançada, cada serviço poderia ter a sua própria instância de base de dados, mas para o âmbito deste projeto achou-se desnecessário.

C. Autenticação com JWT

Decidimos usar JSON Web Tokens porque é o mecanismo sugerido no enunciado e porque torna o sistema *stateless*: o servidor não precisa de guardar sessões, basta verificar a assinatura do token. O token é gerado no login e enviado pelo cliente em cada pedido subsequente no cabeçalho `Authorization: Bearer <token>`.

D. Comunicação entre Serviços

Toda a comunicação é feita de forma síncrona via HTTP REST, seguindo o modelo Request/Response. Consideramos usar comunicação assíncrona com filas de mensagens, mas acabámos por não o fazer porque para o nosso caso de uso (gestão de tarefas) a abordagem síncrona é suficiente e mais simples de implementar e depurar.

V. CONTENTORIZAÇÃO E Deployment

A. Docker

Cada microsserviço tem o seu próprio Dockerfile e corre num contentor independente. A imagem base utilizada é `python:3.11-slim` para os serviços em Python e `nginx:alpine` para o frontend. Usamos também um ficheiro `docker-compose.yml` que permite arrancar todos os 7 contentores (5 serviços + MariaDB + frontend) com um único comando.

A tabela I resume os contentores do sistema.

Tabela I
CONTENTORES DO SISTEMA.

Contentor	Imagem	Porta
orquestrador	python:3.11-slim	5000
servico-autenticacao	python:3.11-slim	5001
servico-utilizadores	python:3.11-slim	5002
servico-tarefas	python:3.11-slim	5003
servico-notificacoes	python:3.11-slim	5004
frontend	nginx:alpine	8080
mariadb	mariadb:10.11	3306

B. Kubernetes com Minikube

Para o *deployment* em Kubernetes, criámos ficheiros YAML para cada componente do sistema. Cada microsserviço tem um *Deployment* (que gere os pods) e um *Service* (que expõe o serviço na rede interna do cluster).

O orquestrador e o frontend utilizam serviços do tipo *NodePort* para serem acessíveis do exterior do cluster. Os restantes serviços usam *ClusterIP*, já que só precisam de ser acedidos internamente pelo orquestrador.

O MariaDB tem associado um *PersistentVolumeClaim* para que os dados não se percam quando o pod reinicia.

Criámos também um script `deploy-minikube.sh` que automatiza todo o processo: inicia o Minikube, constrói as imagens Docker, aplica os manifests e configura o *port-forward* para que o sistema fique acessível a partir do *host*.

VI. FLUXOS PRINCIPAIS

A. Fluxo de Registo e Login

- 1) O cliente envia `POST /api/auth/registar` ao orquestrador com nome, email e senha.
- 2) O orquestrador reencaminha para o serviço de autenticação.
- 3) O serviço de autenticação chama o serviço de utilizadores para criar a conta.
- 4) Se a conta for criada, o serviço de autenticação gera um token JWT e devolve-o ao cliente.
- 5) Em pedidos futuros, o cliente envia o token no cabeçalho.

B. Fluxo de Criação de Tarefa

- 1) O cliente envia `POST /api/tarefas` ao orquestrador com o token e os dados da tarefa.
- 2) O orquestrador valida o token junto do serviço de autenticação.
- 3) Se válido, o orquestrador associa o `utilizador_id` à tarefa e reencaminha para o serviço de tarefas.
- 4) Se a tarefa tem data limite, o orquestrador notifica também o serviço de notificações.
- 5) A resposta é devolvida ao cliente.

VII. ESTRUTURA DO PROJETO

A organização dos ficheiros do projeto é a seguinte:

```
projeto-adm/  
  orquestrador/  
    app.py, Dockerfile, requirements.txt  
  servico-autenticacao/  
    app.py, Dockerfile, requirements.txt  
  servico-utilizadores/  
    app.py, Dockerfile, requirements.txt  
  servico-tarefas/  
    app.py, Dockerfile, requirements.txt  
  servico-notificacoes/  
    app.py, Dockerfile, requirements.txt  
  frontend/  
    index.html, Dockerfile  
  kubernetes/  
    segredos.yaml, mariadb.yaml,  
    orquestrador.yaml, servico-autenticacao.yaml,
```

```
servico-utilizadores.yaml,  
servico-tarefas.yaml,  
servico-notificacoes.yaml,  
frontend.yaml  
scripts/  
  deploy-minikube.sh  
docker-compose.yml  
README.md
```

VIII. TESTES E VALIDAÇÃO

Para validar o sistema, realizámos testes manuais utilizando `curl` e a interface web. Os testes cobriram os seguintes cenários:

- Registo de um novo utilizador e verificação de que o token JWT é devolvido.
- Login com credenciais corretas e incorretas.
- Criação, listagem, edição e eliminação de tarefas.
- Tentativa de aceder a tarefas de outro utilizador (deve devolver erro 403).
- Acesso a rotas protegidas sem token (deve devolver erro 401).
- Verificação do estado de todos os serviços via `/api/saude`.
- Criação de tarefa com data limite e verificação de que a notificação é gerada.
- Marcação de notificação como lida.
- *Deployment* completo em Docker Compose e em Minikube.

Todos os cenários foram testados com sucesso em ambos os ambientes de *deployment*.

IX. DIFICULDADES ENCONTRADAS

Durante o desenvolvimento do projeto, encontrámos algumas dificuldades que vale a pena mencionar:

- **Timing do MariaDB** – Quando os contentores arrancam todos ao mesmo tempo, os serviços em Python tentam ligar-se ao MariaDB antes de este estar pronto. Resolvemos isto com um ciclo de tentativas (*retry loop*) na inicialização de cada serviço.
- **Rede no WSL2** – Como utilizámos Windows com WSL2, o IP do Minikube não era acessível diretamente pelo browser do Windows. A solução passou por usar `kubectrl port-forward` para expor os serviços em `localhost`.
- **CORS** – A interface web faz pedidos a um domínio/porta diferente do orquestrador, o que causava erros de CORS. Resolvemos com a biblioteca `flask-cors` no orquestrador.

X. CONCLUSÃO

O sistema desenvolvido cumpre todos os requisitos mínimos do enunciado: quatro microsserviços independentes (orquestrador, autenticação, utilizadores e tarefas), comunicação REST, orquestração centralizada, contentorização em Docker e integração em Kubernetes via Minikube.

Além disso, implementámos duas funcionalidades extra: um serviço de notificações automáticas e uma interface web funcional.

A principal lição que tirámos deste projeto foi a importância de definir bem os limites de cada serviço antes de começar a programar. A separação em contextos delimitados, tal como foi discutido nas aulas, ajudou-nos a manter os serviços coesos e com pouca dependência entre si.

REFERÊNCIAS

- [1] Conteúdos das aulas teóricas da unidade curricular de Arquitetura e Desenvolvimento de Microsserviços. Disponível em: <https://moodle.ubi.pt/>.
- [2] Documentação da Ferramenta Minikube. Disponível em: <https://minikube.sigs.k8s.io/docs/>.
- [3] S. Newman, “Building Microservices,” O’Reilly Media, 2021.
- [4] D. C. Marinescu, “Cloud Computing: Theory and Practice,” 3.^a Edição, Morgan Kaufmann, 2022.
- [5] L. H. Etkorn, “Introduction to Middleware: Web Services, Object Components, and Cloud Computing,” 1.^a Edição, Chapman and Hall/CRC, 2017.